Theseus

Product and Software Summary

April 2026

Contents

1 Mission and methodological frame **3**

2 Repository at a glance **3**

3 Core softwares **4**
- 3.1 Noosphere — the epistemic engine . . . 4
- 3.2 Dialectic — the live conversation analyzer . . . 5
- 3.3 Founder Portal — the web control plane and desktop app . . . 5
- 3.4 Theseus Public — the publication-only public website . . . 6
- 3.5 Theseus Codex — the firm's living reference . . . 7
- 3.6 Researcher API — the outside-investigator surface . . . 7
- 3.7 Ideological Ontology — research experiments . . . 7

4 Noosphere's internal architecture **8**
- 4.1 The core pipeline (flat modules) . . . 8
- 4.2 The coherence subpackage . . . 9
- 4.3 Subpackages . . . 9

5 Written research catalog **12**

6 Data flow — how a transcript becomes a public conclusion **12**

7 Desktop application and deployment infrastructure **14**
- 7.1 Python app packaging (Dialectic and Noosphere) . . . 14
- 7.2 Electron desktop app (Founder Portal) . . . 14
- 7.3 Docker and web deployment . . . 15
- 7.4 Code signing and notarization . . . 15
- 7.5 CI/CD pipelines . . . 15

8 Operational surface and user roles **16**

9 Evaluation and governance machinery **17**

10 Technology stack **18**

11 Appendix: glossary **18**

1 Mission and methodological frame

Theseus is an intellectual capital firm whose product is *truth-finding methodology*, not truth claims. The distinction is load-bearing: the firm does not claim to know which specific positions on contested questions are correct, but it does claim to build and operate the instruments by which disciplined groups of people converge on better beliefs. That pivot, recorded in `docs/Methodological_Reorientation.pdf`, governs every line of code in this repository.

The conviction underneath that pivot is that the scarce resource in serious knowledge work is not information but *discipline* — the willingness to track one's positions over time, to submit them to adversarial review, to treat contradiction as a signal rather than an inconvenience, to quantify uncertainty honestly, and to publish one's epistemic process rather than just its outputs. Theseus is instrumentation for that discipline.

The pivot has three structural consequences for the codebase:

1. Every judgment the system makes is a *method*, not a one-off function. Methods are versioned, testable against external data, packageable, and documentable in human-readable form. They are the product.

2. Every claim the firm publishes is *traceable* through a cryptographically-signed ledger back to the input artifacts and methods that produced it. Claims without provenance do not leave the building.

3. Every public-facing output passes through a *rigor gate* that enforces provenance, calibrated confidence, peer review, and freshness. Refusals are logged publicly, because the refusal rate itself is part of the methodology's credibility.

2 Repository at a glance

The repository is a monorepo at `~/Desktop/Theseus/` holding six pieces of software, a research and reference corpus, and the operational and distribution infrastructure to build, test, sign, deploy, and install them on macOS and Windows.

Directory	Purpose
`noosphere/`	The core epistemic engine — Python. Ingests transcripts and essays, extracts claims, runs six-layer coherence, builds the knowledge graph, synthesizes conclusions, runs adversarial review and calibration, exposes CLI. Packaged as a standalone binary via PyInstaller. ∼89 Python modules, 95+ test files.
`dialectic/`	Live-conversation analyzer — PyQt6 desktop app. Streams microphone audio through Whisper, segments into claims, flags contradictions on a dashboard in real time. Distributed as a macOS `.app`/DMG and Windows `.exe`/NSIS installer. 11 modules, 5 test files.

Directory	Purpose
`founder-portal/`	Founders' web control plane and Electron desktop app — Next.js 16 / React 19 / Prisma 7. Authentication, artifact upload, ingestion dispatch, conclusion review, adversarial page, decay dashboard, rigor-gate queue. Distributed as a macOS/Windows Electron app (via `electron-builder`) and as a Docker container. 50+ TS/TSX files.
`theseus-public/`	Publication-only public website — Next.js static export. Signed method docs, MIP registry, per-conclusion provenance, methodology refusal dashboard. Deployed via Docker/nginx, Vercel, or Netlify. 18 files.
`theseus-codex/`	Firm reference documents and project memory — Next.js. A living internal wiki with Oracle chat, two-factor auth, and contribution workflows.
`researcher_api/`	Rate-limited HTTP API for outside investigators. Claim extraction, coherence, embedding, calibration, replay — on researcher-supplied text only; no access to firm data.
`ideologicalOntology/`	Research experiments in contradiction geometry (Embedding Geometry Conjecture, Reverse Marxism flipping framework, experiments log). Primarily data + analysis scripts.
`docs/`	Published PDFs of research and architecture, plus living operational markdowns and per-area specs (API methods, researcher API, red-team, peer review, interop, eval).
`.github/`	GitHub Actions CI/CD: cross-platform desktop builds (macOS/Windows), Docker image builds, release orchestration, round-3 invariant checks, red-team workflows, Dependabot.
`deploy/`	Docker Compose stacks and Kubernetes Helm charts for the web services.
`scripts/`	Shared build infrastructure: macOS code signing (`codesign_macos.sh`), Apple notarization (`notarize_macos.sh`), Windows Authenticode signing (`codesign_windows.ps1`), and 10+ invariant-check lint scripts.
`packages/`	Shared TypeScript packages (`theseus-api-types`) cross-used by the web frontends.
`Podcast talks/`	Raw source transcripts of weekly 4-hour founder discussions.

3 Core softwares

3.1 Noosphere — the epistemic engine

Noosphere is the Brain of the Firm. It takes the weekly four-hour podcast transcripts (plus founder essays, memos, and Dialectic session exports) as input, decomposes them into atomic claims, evaluates those claims across multiple coherence methods, tracks principles and their evolution over time, and synthesizes firm-tier conclusions with calibrated confidence. Every judgment it makes is a registered *method* routed through a central registry; every method invocation is recorded in an append-only audit ledger; every evidence flow between claims and conclusions is represented as a typed cascade graph.

The engine is executable from a Typer CLI (`typer_cli.py`) and a legacy Click CLI (`cli.py`) with commands covering ingest, synthesis, graph inspection, adversarial, calibration, research-advisor output, backup, restore, method management, ledger verification, cascade diagnostics, evaluation, decay scheduling, peer review, interop packaging, rigor-gate submission, and inverse inference. It speaks to a SQLite (or Postgres) store for claims, invocations, ledger entries, and review reports, and to an on-disk data directory for embeddings, the ontology graph, and synthesized artifacts.

Distribution. Noosphere is packaged as a standalone CLI binary via PyInstaller. The `noosphere.spec` bundles Alembic migration scripts and all Python dependencies into a single-directory distribution. ML model weights (sentence-transformers, spaCy) are downloaded on first run rather than bundled, keeping the distributable under reasonable size. A `frozen_support.py` module resolves paths correctly in both development and frozen (PyInstaller) modes. Platform build scripts produce a macOS `.tar.gz` archive and a Windows NSIS installer that adds Noosphere to the system PATH. A background update checker (`updater.py`) polls a version manifest URL and prints a Rich-formatted notification when a newer release is available.

3.2 Dialectic — the live conversation analyzer

Dialectic listens to a discussion while it is happening. It opens a microphone stream via `sounddevice`, runs voice-activity detection to cut the audio into speech segments, and pushes those segments to a `faster-whisper` transcription worker. Finalized utterances are claim-segmented with speaker attribution, embedded with sentence-transformers MiniLM, clustered for topic drift, and run pairwise through a DeBERTa-v3 NLI head to surface contradictions. A PyQt6 dashboard renders the transcript, the claim graph, and contradiction alerts with under two seconds of latency on Apple Silicon or a laptop GPU.

A live interlocutor mode (SP09) lets Dialectic surface short, third-person prompts — "this contradicts your position in episode 47," "an open question has gone untouched," "what criterion would falsify this prediction?" — under strict consent and a per-session budget. Modes range from silent (default) through passive overlay, conversational with optional TTS, and tutor (higher rate, requires acknowledgment). Every candidate passes a quality gate; every intervention and drop is logged for post-session review.

Saved sessions are JSON Lines, one line per finalized claim with timestamp, speaker, text, embedding, and contradictions, and are the native input format for Noosphere's ingester.

Distribution. Dialectic is packaged as a standalone desktop application via PyInstaller. The `dialectic.spec` bundles PyQt6, sounddevice, faster-whisper, sentence-transformers, torch, and all supporting libraries into a single-directory distribution. A `resources.py` module provides frozen-mode-aware path resolution for assets, configuration, and user data directories (platform-specific: `~/Library/Application Support/Dialectic` on macOS, `%APPDATA%\Dialectic` on Windows). Build scripts produce a macOS `.app` bundle in a DMG disk image with a drag-to-Applications layout, and a Windows NSIS installer with Start Menu and Desktop shortcuts. A threaded background update checker (`updater.py`) polls a version manifest and notifies the user within the Qt dashboard when a newer release is available.

3.3 Founder Portal — the web control plane and desktop app

The Founder Portal is where the firm's founders operate Noosphere without touching the command line. It handles authentication against seeded founder accounts with bcrypt-hashed credentials, file

upload and asynchronous ingestion, live streaming of ingestion logs, conclusion review, contradiction triage, the adversarial challenge flow, decay dashboard, peer review tab, rigor-gate submission queue, provenance exploration, and the cascade viewer.

The stack is Next.js 16 (App Router) on React 19 with Prisma 7 over SQLite. Operationally, the portal points at the same SQLite file that Noosphere writes so the `adversarial_challenge` and related tables are shared directly; there is no API marshalling layer for internal reads. Mutations that write to publicly-visible state go through a Python bridge (`src/lib/ingest.ts`) that shells out to the Noosphere CLI under the portal user's environment, streaming stdout back into the database for live progress.

The portal's `src/app/` contains dedicated pages for: `dashboard`, `upload`, `conclusions`, `cascade`, `provenance`, `eval` (including `eval/runs`), `post-mortem`, `peer-review`, `decay`, `rigor-gate`, `methods`, `founders`, `sessions/[id]/reflection`, and `voices/[id]`.

Distribution. The Founder Portal ships as both a web service and a desktop application.

Desktop (Electron). An Electron shell (`desktop/main.ts`) wraps the Next.js standalone build. On launch, it resolves a platform-appropriate SQLite database path via `db-path.ts`, runs Prisma migrations, starts the Next.js standalone server as a child process via `next-server.ts` (polling for readiness with exponential backoff), and opens a `BrowserWindow` pointed at the local server. The window uses `titleBarStyle: hiddenInset` on macOS for a native look, enforces `contextIsolation: true` and `nodeIntegration: false`, and opens external links in the system browser. Single-instance lock prevents duplicate launches. An auto-updater (`desktop/updater.ts`) integrates `electron-updater` to check GitHub Releases for new versions, download in the background, and prompt the user to restart. The `electron-builder.yml` configuration produces a macOS DMG (universal architecture, hardened runtime, entitlements for JIT and network access) and a Windows NSIS installer (customizable install directory, Start Menu and Desktop shortcuts). A development helper (`desktop/dev.ts`) starts the Next.js dev server and Electron concurrently for local iteration.

Web (Docker). A multi-stage Dockerfile builds the Next.js standalone output, generates the Prisma client, pre-runs migrations to produce a seed database, and serves from a minimal `node:20-alpine` image. The SQLite database is mounted as a Docker volume for persistence.

3.4 Theseus Public — the publication-only public website

The public site is the firm's outward intellectual face. It reads only from the publication store; it has no write paths. Each conclusion published here carries a provenance block that links to the ledger entries that produced it, a sanitized adversarial history showing reviewer roles (but never agent identities), and the corpus hash at the time of publication. The site also hosts:

- `/methods/` — the public registry of packaged, signed method documentation bundles.
- `/interop/` — published Methodology Interoperability Packages (MIPs) for external adoption.
- `/methodology/rigor/` — monthly pass/fail rates of the publication rigor gate.
- `/methodology/overrides/` — every founder override of the rigor gate, with full justification.
- `/methodology/decay/` — aggregated stats on what was retired and why.
- `/open-questions/` and `/responses/` — active unresolved topics and the firm's structured replies to external critiques.

Writes to the public content store bypassing the rigor gate are architecturally impossible; a storage-layer assertion rejects un-gated writes, and a CI lint verifies that every publication handler wears the `@gated` decorator.

Distribution. The public site uses Next.js static export (`output: "export"`). A multi-stage Dockerfile builds the static output and serves it via `nginx:1.27-alpine` with gzip compression and security headers. Deployment configurations are provided for Vercel (`vercel.json`) and Netlify (`netlify.toml`).

3.5 Theseus Codex — the firm's living reference

The Codex is the firm's internal reference wiki. It holds the prior research artifacts the founders want to preserve and re-ingest over time: methodological writings, technical experiments, historical essays, and project memory. It is a Next.js application backed by SQLite via Prisma, featuring full-text search over codex entries, a contribution workflow for new entries, an Oracle chat interface (Claude integration), user profiles, attachment support, and two-factor authentication. Its role in the pipeline is as a structured corpus that Noosphere periodically re-ingests so that the engine's knowledge graph is constantly up to date with the firm's own canon.

3.6 Researcher API — the outside-investigator surface

The Researcher API (SP07) is a rate-limited, authenticated HTTP service intended for academic or industrial researchers who want to apply Theseus methods to their own data without access to the firm's internal corpus. It exposes claim extraction, coherence scoring, embedding, calibration scoring, and a stub replay endpoint — all restricted to researcher-supplied text. It is deployed as a separate FastAPI service with its own API-key registry, privacy policy, security documentation, acceptable-use policy, and citation guidance.

3.7 Ideological Ontology — research experiments

This directory holds the empirical work that validates the firm's methodological claims. It contains the Embedding Geometry Conjecture's validation experiments, the Reverse Marxism ideology-flipping framework, and the catalog of contradiction-geometry experiments with their methods and results. These are not deployed software; they are the experimental substrate that justifies why the coherence engine is shaped the way it is.

4 Noosphere's internal architecture

Noosphere is not a monolith. It is a composition of approximately 15 subpackages, each responsible for a clean slice of the epistemic pipeline, plus ∼40 flat modules at the top level. This section walks the live package tree under `noosphere/noosphere/`.

4.1 The core pipeline (flat modules)

These are single-file modules at the top of `noosphere/noosphere/` that implement the foundational pipeline. Many have been ported into the method registry as registered methods; the flat-module originals remain as the canonical implementations.

Module	Responsibility
`models.py`	Pydantic types for every cross-boundary object: `Artifact`, `Chunk`, `Claim`, `Principle`, `Conclusion`, `DriftEvent`, `ResearchSuggestion`, and all round-3 types (Method, MethodInvocation, LedgerEntry, CascadeNode/Edge, DecayPolicy, ReviewReport, RigorVerdict, etc.).
`store.py`	SQLModel-backed persistence; typed CRUD; chain-integrity enforcement for ledger entries and cascade edges.
`ids.py`	Deterministic content-addressed identifiers for every domain object.
`config.py`	Pydantic-settings configuration, reading environment variables and optional `theseus.toml`.
`observability.py`	Structlog JSON logger with per-orchestrator-run correlation IDs.
`llm.py`	LLM client protocol + Anthropic/OpenAI implementations + mock for tests.
`embeddings.py`	Embedding client protocol + sentence-transformers implementation + on-disk cache.
`ingester.py`	Markdown, plain-text, and WebVTT transcript ingest with YAML frontmatter, speaker-turn segmentation, and Dialectic session import.
`claim_extractor.py`	Chunk-to-claim extraction via LLM with structured JSON output; cached per chunk.
`classifier.py`	Zero-shot MNLI verification of claim type; flags disagreements between LLM-assigned and classifier-assigned labels.
`embed_pass.py`	Orchestrated embedding pass over claims lacking embeddings; batched; supports rebuild.
`ontology.py`	Growing topic ontology via HDBSCAN on UMAP-reduced embeddings; persistent cluster identities; spaCy-based entity linking.
`geometry.py`	Contradiction-geometry detection via Householder reflection and difference-vector sparsity per the Embedding Geometry Conjecture.
`conclusions.py`	Synthesizes firm-tier, founder-tier, and open-tier conclusions with tiered confidence discount.
`synthesis.py`	End-to-end synthesis pipeline tying ingest, extraction, coherence, and conclusion into a single runner.
`adversarial.py`	Generates structured objections to firm-level conclusions, runs them through the six-layer coherence stack, and persists verdicts.

Module	Responsibility
`redteam.py`	Red-team prompts and harness for adversarial robustness testing.
`meta_analysis.py`	Five-criterion meta-analysis gate over candidate conclusions (severity, coherence, freshness, provenance, calibration).
`predictive_extractor.py`	Extracts falsifiable predictions with resolution criteria.
`resolution.py`	Resolves predictions against observed outcomes.
`scoring.py`	Calibration-scoreboard computation (Brier, log loss, ECE) over resolved predictions.
`retrieval.py`	Embedding-based corpus retrieval for downstream methods and the research advisor.
`founders.py`	Voice decomposition — separates named founder positions from firm-tier claims.
`voices.py`	Voice-specific retrieval and aggregation utilities.
`literature.py`	External-literature search and claim-match scoring against the firm's corpus.
`research_advisor.py`	Generates ranked research suggestions based on open questions and blindspots.
`temporal.py`	Tracks principle emergence, drift, and strengthening across episodes; produces evolution reports.
`temporal_replay.py`	Counterfactual temporal replay — re-runs methods against historical corpus slices.
`tenancy.py`	Multi-tenant partitioning for cloud deployments; scopes every read/write by tenant id.
`storage_client.py`	Object-store abstraction for input/output blobs referenced by the ledger.
`backup_restore.py`	Archive-and-restore of SQLite + data directory.
`orchestrator.py`	Coordinates multi-step pipelines (ingest → extract → embed → cohere → synthesize).
`frozen_support.py`	Path resolution for PyInstaller frozen mode (bundle dir, data dir, alembic dir).
`updater.py`	Background version checker against a release manifest URL; Rich-formatted CLI notification.
`typer_cli.py`	Primary Typer CLI entry point with subcommand auto-discovery.
`__main__.py`	Module entry point for `python -m noosphere`.

4.2 The coherence subpackage

`noosphere/noosphere/coherence/` holds the six-layer coherence stack that is Theseus's principal contribution to how one measures belief-set consistency. Each layer is independent and has a clear empirical semantics.

The aggregator is deliberately conservative: when layers disagree past a threshold, the output is `unresolved` rather than a point verdict. Unresolved is a first-class output in every downstream consumer, not an error state.

4.3 Subpackages

Layer	What it measures
`nli.py`	Natural-language inference verdicts (entail / neutral / contradict probabilities) via a DeBERTa-v3 MNLI/ANLI head, batched.
`argumentation.py`	Formal argumentation-framework analysis (Dung-style admissibility, preferred / grounded extensions).
`probabilistic.py`	Probabilistic-consistency check: is the claim set jointly satisfiable under any coherent probability assignment?
`geometry.py`	Embedding-geometry test using the validated Embedding Geometry Conjecture.
`information.py`	Information-theoretic compressibility: claim-set entropy and redundancy.
`judge.py`	LLM-as-judge over the full pair with structured output and disagreement-with-other-layers flagging.
`aggregator.py`	Combines the six layers into a `SixLayerScore` with explicit `unresolved` output when layers disagree materially.
`engine.py`	Orchestrates a coherence run end-to-end; populates the coherence cache.
`scheduler.py`	Picks pairs to score, prioritizing load-bearing-for-conclusions pairs.
`calibration.py`	Maintains calibration-bundles of reference pairs with gold labels, used to recalibrate layer weights on model upgrades.
`metrics.py`	Macro-F1 and per-layer agreement metrics for regression testing.
`cache.py`	On-disk cache of per-pair verdicts, invalidated on model-version change.

Package	Role
`methods/`	Central registry of registered methods with the `@register_method` decorator, its pre/post-hook API, the `_legacy/` pre-port copies for parity testing, and the ported implementations: `nli_scorer`, `contradiction_geometry`, `six_layer_coherence`, `suggest_research`, `extract_claims`, `classify_claim_type`, `synthesize_conclusion`, `decompose_voice`, `external_claim_match`, `extract_from_corpus`, `extract_prediction`.
`ledger/`	Append-only, Ed25519-signed, Merkle-chained audit log. Every method invocation produces one entry. Externally verifiable via `theseus ledger verify` and exportable as a self-contained audit bundle. Modules: keys, ledger, export, verify, hooks.
`cascade/`	Typed evidence-flow graph linking artifacts → chunks → claims → principles → conclusions with eleven relation kinds. Supports proof-bundle export, cut-reports, single-point-of-failure diagnostics, cycle detection, and rendering. Modules: graph, traverse, render, export, diagnostics, hooks.
`evaluation/`	Counterfactual calibration rig. `CorpusSlicer` presents a read-only view of the store pinned to a past `as_of`; methods invoked against it raise `EmbargoViolation` on out-of-slice reads. Computes Brier, log loss, ECE, reliability curves per method. Modules: slicer, outcomes, metrics, counterfactual, quality assessment.
`inference/`	Inverse inference engine. Given a resolved event, back-propagates to corpus claims that would have predicted it; produces a structured blindspot report (missing entities, missing mechanisms, adjacent-empty topics).

Package	Role
`external_battery/`	Benchmarks methods against external corpora (Good Judgment Project, Metaculus, ClaimReview-tagged fact-checking, replication projects). Ships with a five-way failure taxonomy and per-corpus calibration metrics.
`peer_review/`	Adversarial peer-review swarm with seven declared reviewer roles (methodological, evidential, statistical, adversarial-literature, replication, rhetorical, humility). Enforces rebuttal for every major or blocker finding; tracks reviewer-level calibration discount factors.
`decay/`	Policy-driven re-validation of stored claims and conclusions. Policies include fixed-interval, evidence-changed, method-version-bump, embedding-drift, outcome-observed, and calibration-regression. Freshness is always derived, never cached. Retirement is irreversible.
`rigor_gate/`	Publication choke point. Every submission to the public layer runs through nine declared checks; failures block; founder overrides are public and ledger-recorded. Refusal rates are published monthly.
`transfer/`	Packages a registered method as a Docker-reproducible, signed artifact with spec, rationale, implementation, adapter, eval card, and transfer-degradation studies.
`docgen/`	Compiles signed, versioned `MethodDoc` bundles from spec + rationale + implementation + evaluation data. AST-diff-based changelog enforcement on version bumps.
`interop/`	Builds Methodology Interoperability Packages (MIPs) — distributable bundles of methods + docs + cascade schema + gate checks + workflow runner for external adoption. Ships with a minimal non-Turing-complete YAML workflow language.
`mitigations/`	Security and robustness modules: ingestion guard, citation anchor, tenant audit, calibration guard, temporal guard, embedding-text validation, coherence judge guard. Consumed by the red-team harness.
`cli_commands/`	Typer subcommand modules auto-discovered by the main CLI. One file per subsystem.

5 Written research catalog

The PDFs in `docs/` are the firm's intellectual justification for why the software is shaped the way it is. They are not vestigial; every one of them either constrains an implementation decision or is cited by one.

Document	What it covers
`Methodological_Reorientation.pdf`	The pivot. Defines why Theseus's product is methodology, not claims. Governs every downstream architectural choice.
`The_Meta_Method.pdf`	The firm's methodological commitments in detail. What disciplined belief-revision looks like and what it costs.
`Product_Description.pdf`	How the three user-facing softwares (Noosphere, Dialectic, Founder Portal) fit together. The integration narrative.
`Noosphere_Project_Status.pdf`	Engineering status snapshot. What is implemented, what is pending, where the seams are.
`Server_Architecture.pdf`	Deployment architecture, including the multi-tenant cloud layout and the split between public and private services.
`Geometry_of_Unresolution.pdf`	Mathematical treatment of epistemic unresolvability. The formal substrate behind why `unresolved` is a first-class output.
`Podcast_Infrastructure_Research.pdf`	Research motivating the Dialectic audio pipeline and the Noosphere ingester.
`Transcript_Extraction_Research.pdf`	Research motivating the chunk-to-claim extraction pipeline and its tradeoffs.
`Founders_Reading_and_Research.pdf`	Reading list and background material shared among founders.
`Founders_Questions.pdf`	Active open questions the firm is tracking.
`Performance_Synthesis.pdf`	Method performance and synthesis strategies.
`Voices_Method.pdf`	Voice-decomposition methodology for separating individual founder positions.

Living operational markdowns in `docs/` include `Operations_Manual.md` (daily workflow, failure modes, backup/restore, adversarial protocol, calibration scoreboard), `Robustness_Ledger.md` (known-attack log and mitigations), and per-area subdirectories: `api-methods/` (extract-claims, embed, predict-score, coherence, replay), `researcher-api/` (privacy, security, acceptable use, citation), `redteam-methods/`, `peer_review/` (reviewer specs), `interop/` (MIP specification), `eval/` (external corpora), and `methods/`.

6 Data flow — how a transcript becomes a public conclusion

The canonical path through the system is the weekly transcript → public conclusion path. It composes every subsystem and is therefore the most informative flow to narrate.

1. **Capture.** Either the founders record a conversation, in which case Dialectic runs live and exports a JSONL session file at completion, or an existing transcript is placed into `Podcast talks/`.

2. **Ingest.** A founder uploads the file via the Founder Portal or runs `python -m noosphere ingest`. The ingester parses the file, extracts frontmatter, segments into chunks on paragraph or speaker-turn boundaries, and writes `Artifact` and `Chunk` rows. Content-addressed IDs make the ingest idempotent.

3. **Claim extraction.** `claim_extractor.py` walks each chunk through the LLM client with a structured JSON output prompt, producing zero or more `Claim` rows with type (empirical / normative / methodological / predictive / definitional), confidence-hedge phrases, and evidence pointers. `classifier.py` re-verifies the type label with zero-shot MNLI; disagreements are flagged.

4. **Embedding.** `embed_pass.py` batches new claims and writes sentence-transformers embeddings to the on-disk cache and `embeddings` table.

5. **Ontology update.** `ontology.py` assigns each new claim to a topic cluster via cosine-nearest-centroid on UMAP-reduced embeddings, with HDBSCAN expansion when the cluster drift exceeds a threshold. Entity-linking collapses name variants to canonical `Entity` rows.

6. **Coherence scoring.** The `coherence/` scheduler picks pairs of claims to score, prioritizing pairs that are load-bearing for existing firm-tier conclusions. Each pair runs through all six layers; the aggregator combines them into a `SixLayerScore` with explicit `unresolved` output on disagreement.

7. **Cascade emission.** Each registered method invocation emits cascade edges via the post-hook registered by `cascade/hooks.py`: `extracted_from`, `coheres_with`, `contradicts`, `predicts`, `supports`, `refutes`, `depends_on`, etc.

8. **Ledger write.** Concurrently, `ledger/hooks.py` writes a signed ledger entry capturing the method, its input hash, its output hash, and pointers to the stored input and output blobs.

9. **Synthesis.** `synthesis.py` composes `run_synthesis_pipeline`: it promotes candidate principles from clusters, attaches supporting claims, runs the five-criterion meta-analysis, and persists firm-tier / founder-tier / open-tier `Conclusion` rows.

10. **Adversarial review.** `adversarial.py` generates structured objections to firm-tier conclusions, scores them through the same coherence stack, and writes `adversarial_challenge` rows. Fallen challenges force demotion unless shadow mode is set.

11. **Peer-review swarm.** On transition to firm tier, a conclusion auto-queues a swarm review across seven reviewer roles, each producing a `ReviewReport`. Major or blocker findings require structured rebuttals before the conclusion can advance.

12. **Decay binding.** Each new conclusion is bound to a default decay policy (typically `FixedInterval(days=30)` for firm-tier). The scheduler will re-validate on a cadence and demote to open tier if re-validation disagrees.

13. **Rigor gate.** When the founder elects to publish, the conclusion is submitted to `rigor_gate/`. Nine declared checks run in parallel; any blocker fails the submission; pass-with-conditions attaches machine-readable publication conditions; pass allows publication. Every verdict is ledger-recorded; founder overrides are public.

14. **Publication.** On pass, the conclusion lands in the public content store. The public site reads from that store only; writes bypassing the gate are architecturally impossible.

15. **Re-ingestion.** The Codex and published conclusions are periodically re-ingested so that the firm's own output becomes part of its epistemic substrate for future rounds.

7 Desktop application and deployment infrastructure

Every piece of Theseus software is distributable as an installable application or deployable service. The build, signing, and distribution infrastructure was added in round 4.

7.1 Python app packaging (Dialectic and Noosphere)

Both Python applications use PyInstaller to produce standalone executables that run without requiring a Python installation. Each has:

- A `.spec` file defining the PyInstaller analysis: hidden imports for all transitive dependencies (PyQt6, torch, sentence-transformers, faster-whisper, etc.), data bundles (assets, Alembic migrations), exclusions (tensorflow, jupyter, pytest), and platform-conditional targets (`.app` BUNDLE on macOS for Dialectic; console-mode directory on both platforms for Noosphere).
- A `resources.py` or `frozen_support.py` module that resolves paths correctly in both development and frozen modes, with platform-specific user data directories.
- `scripts/generate_icons.py` that produces macOS `.icns` and Windows `.ico` from a source PNG using Pillow and (on macOS) `iconutil`.
- `scripts/build_macos.sh` and `scripts/build_windows.ps1` that orchestrate the full build: icon generation, PyInstaller execution, and platform-specific packaging (DMG via `create-dmg`/`hdiutil` on macOS; NSIS installer on Windows).
- `installer/` containing the NSIS script (`.nsi`) for Windows installer creation (with Start Menu shortcuts, Desktop shortcut, uninstaller, registry entries) and `dmg-settings.json` for DMG layout.
- An `updater.py` module providing a non-blocking background update checker that polls a JSON version manifest and notifies the user when a newer release is available.

Single-directory mode is used rather than single-file because the ML dependency stack (torch, CTranslate2, sentence-transformers) is too large for single-file extraction to be practical.

7.2 Electron desktop app (Founder Portal)

The Founder Portal's Electron wrapper comprises six TypeScript modules in `desktop/`:

- `main.ts` — app lifecycle, single-instance lock, BrowserWindow creation with context isolation, external-link handling.
- `next-server.ts` — starts the Next.js standalone server as a `fork`'d child process on a free port, with readiness polling (exponential backoff, 30s timeout) and graceful shutdown (SIGTERM then SIGKILL after 5s).
- `db-path.ts` — resolves SQLite to the platform-appropriate app-data directory (`userData/data/founder-portal`

- `preload.ts` — minimal context bridge exposing platform info and deep-link IPC.
- `updater.ts` — `electron-updater` integration: silent check on launch, user-prompted restart on download completion, error logging via `electron-log`.
- `dev.ts` — development helper that starts `next dev` and Electron concurrently.

The `electron-builder.yml` configuration produces: macOS DMG (universal architecture, hardened runtime, `entitlements.mac.plist` granting JIT, network, and file access) and Windows NSIS installer (customizable directory, Start Menu category "Theseus"). Auto-update publishes to GitHub Releases.

7.3 Docker and web deployment

- `founder-portal/Dockerfile` — multi-stage build: dependency install, Next.js build with Prisma generation, standalone extraction, seed database from pre-run migrations, `node:20-alpine` runner with volume-mounted SQLite.
- `theseus-public/Dockerfile` — multi-stage: Node build to static export, `nginx:1.27-alpine` serving with gzip, `try_files` for clean URLs, security headers.
- `docker-compose.yml` (repo root) — Founder Portal on port 3000, Public on port 8080, shared `portal-data` volume for SQLite persistence.
- `deploy/compose/` — extended compose configuration for production stacks.
- `deploy/helm/theseus/` — Kubernetes Helm chart with templates for portal and ingest-worker deployments.
- `theseus-public/vercel.json` and `theseus-public/netlify.toml` — managed hosting configurations with security headers.

7.4 Code signing and notarization

Three shared scripts in `scripts/` handle signing across all desktop applications:

- `codesign_macos.sh` — deep-signs all nested binaries (`.dylib`, `.so`, executables) with a Developer ID certificate, sets hardened runtime and timestamp. Gracefully skips when no signing identity is provided.
- `notarize_macos.sh` — submits DMG or ZIP to Apple's notarization service via `xcrun notarytool`, waits up to 600s, and staples the ticket on success. Gracefully skips when credentials are absent.
- `codesign_windows.ps1` — signs all `.exe` and `.dll` files with an Authenticode certificate via `signtool`, using SHA-256 and DigiCert timestamping. Gracefully skips when no certificate is provided.

All three scripts are designed for CI integration: they exit 0 with a warning when credentials are not configured, ensuring unsigned development builds succeed.

7.5 CI/CD pipelines

Six GitHub Actions workflows in `.github/workflows/` automate the full build-test-sign-release lifecycle:

- `build-dialectic.yml` — matrix build on macOS 14 (ARM) and Windows. Python 3.11 setup, pip install, PyInstaller build, DMG creation (macOS), code signing and notarization when secrets are configured. Uploads platform artifacts.
- `build-noosphere.yml` — same matrix structure for the Noosphere CLI. Produces tar.gz (macOS) and directory (Windows).
- `build-founder-portal.yml` — matrix build for Electron. Node 20 setup, Next.js build, TypeScript compilation, icon generation, `electron-builder` with platform flag. Code signing via certificate secrets (macOS: P12 base64 + password; Windows: PFX base64 + password).
- `build-docker.yml` — builds Docker images for Founder Portal and Theseus Public using Buildx with GitHub Actions cache. Validates `docker-compose.yml`.
- `release.yml` — triggered on `v*` tags. Calls the three build workflows, downloads all artifacts, and creates a draft GitHub Release via `softprops/action-gh-release` with auto-generated release notes.
- `round3.yml` — invariant checks and system tests for round-3 architectural constraints.
- `noosphere-redteam.yml` — adversarial/security testing workflow.

`.github/dependabot.yml` configures weekly dependency updates for npm (Founder Portal, Public), pip (Noosphere, Dialectic), and GitHub Actions.

All build workflows accept `workflow_call` for reuse by the release orchestrator, plus `push` (path-filtered to their directory), `pull_request`, and `workflow_dispatch` triggers.

8 Operational surface and user roles

Three roles interact with the system, each through a distinct surface.

Founder. The primary user. Works through the Founder Portal (web UI or Electron desktop app, credentialed) and secondarily through the CLI when diagnosing. Uploads transcripts and essays, reviews synthesized conclusions, triages contradictions and adversarial challenges, authorizes rigor-gate overrides, retires invalidated objects. Every founder action is ledger-recorded under an `Actor(kind="human")`.

Researcher (external). An outside academic or industrial user. Accesses the Researcher API via HTTP with an issued API key. Can submit their own text to claim extraction, coherence scoring, embedding, calibration scoring, and replay endpoints. Cannot touch firm-internal data. Can download packaged methods and MIPs from the public site and re-run them locally against their own corpus.

Public reader. Anyone on the internet. Reads the public site: published conclusions with full provenance, adversarial histories, method documentation, MIP registry, methodology refusal dashboards, open questions, and the firm's structured responses to external critiques. Cannot write; the architecture forbids it.

The operational surface for the firm itself is the CLI:

- `noosphere ingest` / `synthesize` / `research` — the core pipeline commands.
- `noosphere methods` — list, show, run, diff, package, document, extract-candidates.

- `noosphere ledger` — verify the chain, export an audit bundle.
- `noosphere cascade` — explain, cut, export a proof bundle, diagnostics.
- `noosphere eval counterfactual / external` — run calibration and external-battery benchmarks.
- `noosphere inverse` — post-mortem on a resolved event.
- `noosphere review` — swarm, rebuttals-pending, reviewer-calibration.
- `noosphere decay` — status, revalidate, schedule, retire.
- `noosphere interop` — build, run, scaffold, submit-transfer.
- `noosphere gate` — submit, status, override, refusal-report.
- `noosphere backup` / `restore` — archive and restore full state.
- `noosphere adversarial` — generate and evaluate adversarial challenges.

9 Evaluation and governance machinery

Seven interlocking mechanisms collectively make Theseus's methodological claims falsifiable.

Six-layer coherence. Every pair of claims load-bearing for a firm conclusion is scored on all six layers. Disagreement among layers yields `unresolved`, which blocks promotion to firm tier.

Adversarial coherence. Every firm-tier conclusion is subjected to structured LLM-generated objections that are themselves scored by the same coherence stack. Fallen challenges demote the conclusion unless addressed by a human reviewer with documented pointer.

Peer-review swarm. Seven declared reviewer roles with explicit bias profiles examine each firm-tier conclusion independently. Major and blocker findings require structured rebuttals. Reviewer-level calibration is tracked; poorly-calibrated reviewers see their findings discounted, never dropped.

Calibration scoreboard. Every falsifiable prediction the firm makes is tracked with crisp resolution criteria. Brier, log loss, and ECE are published per-method per-month. Confidence claims in public outputs are discounted by the producing method's calibration history.

Counterfactual evaluation. Each registered method is re-run against historical corpus slices to measure what it would have concluded at prior dates; the results are compared against observed outcomes. Temporal leakage is detected via a canary-claim test that fails the run if any post-cut material appears in method output.

External corpus battery. Methods are benchmarked monthly against external corpora (Good Judgment, Metaculus, ClaimReview, replication projects). Per-corpus calibration metrics are published alongside a five-way failure taxonomy. Methods that regress past a threshold trigger a red-flag ticket.

Rigor gate. Every public-facing output runs through nine declared checks before publication. Refusal rates, override counts, and override justifications are all public. A 100% pass rate for multiple consecutive months would be a sign that the checks are too weak, not that the firm is flawless.

All seven mechanisms write to the same append-only ledger, producing a single chronological record from which external auditors can reconstruct every epistemic judgment the firm has ever made.

10 Technology stack

Layer	Technologies
Backend/Engine	Python 3.11: sentence-transformers, transformers, spaCy, scikit-learn, HDBSCAN, UMAP, NetworkX, PyNaCl (Ed25519), zstandard, z3-solver. SQLModel + Alembic over SQLite or PostgreSQL. LLM clients: Anthropic Claude, OpenAI.
Frontends	Next.js 16 (App Router), React 19, TypeScript 5, TailwindCSS 4. Prisma 7 ORM with better-sqlite3.
Desktop	Electron 35 with electron-builder 26 and electron-updater (Founder Portal). PyQt6 with pyqtgraph and qasync (Dialectic). PyInstaller for all Python apps.
Audio/NLP	faster-whisper (CTranslate2), sounddevice (PortAudio), DeBERTa-v3 NLI, sentence-transformers MiniLM.
Deployment	Docker multi-stage (node:20-alpine, nginx:1.27-alpine), Docker Compose, Kubernetes Helm charts. Vercel/Netlify for static hosting.
CI/CD	GitHub Actions on macOS 14 (ARM) and Windows-latest. Code signing: macOS `codesign` + `notarytool`, Windows `signtool` Authenticode. Dependabot for dependency updates.
Testing	pytest (Python), Vitest (TypeScript), structured invariant-check scripts, red-team harness, calibration regression suites.

11 Appendix: glossary

Artifact A raw source document ingested into the system (transcript, essay, memo, Dialectic session).

Chunk A sub-segment of an artifact on paragraph or speaker-turn boundaries.

Claim An atomic proposition extracted from a chunk, with type (empirical / normative / methodological / predictive / definitional), confidence hedges, and evidence pointers.

Conclusion A synthesized firm-level statement derived from supporting claims through the coherence and meta-analysis pipeline, carrying a tier (firm / founder / open) and calibration-discounted confidence.

Coherence A six-layer measure of consistency across a set of claims, yielding either a point verdict or `unresolved`.

Cascade The typed directed graph of evidence-flow relations between artifacts, chunks, claims, principles, and conclusions.

Ledger The append-only, signed, Merkle-chained record of every method invocation and human epistemic action.

Method A versioned, registered judgment function with declared input/output schemas, rationale, and dependencies.

MethodDoc A signed, publishable documentation bundle compiled from a method's spec, rationale, implementation, and evaluation results.

MIP Methodology Interoperability Package — a distributable bundle of methods and their supporting artifacts for external adoption.

Rigor Gate The publication choke point; every submission to the public layer passes through nine checks.

Decay The policy-driven re-validation of stored objects; objects that fail re-validation are retired, never deleted.

Freshness The derived state of a stored object ($\in$ {fresh, aging, stale, retired}), computed from last-validation and bound policies.

Unresolved A first-class judgment output indicating honest epistemic uncertainty rather than an error.

Voice A named founder's position on a claim, tracked separately from firm-tier positions.

Principle A clustered abstraction over supporting claims, eligible for conclusion promotion if it meets the five-criterion meta-analysis.

Blindspot A structured report of what a resolved event implied that the corpus did not cover.

Frozen Mode PyInstaller's runtime state where the application runs from a bundled directory rather than source; path resolution adapts accordingly.

Standalone Build Next.js output mode that produces a self-contained server directory, used by both the Electron wrapper and Docker container.